DAY 2

PAGE No.
DATE / /

Dry run.

Array Basics.

```
int[] marks = new int[5];
```

marks

Index :	0	1	2	3	4
value :	0	0	0	0	0

```
String [] names = new String [3];
```

names

Index :	0	1	2
value :	null	null	null.

int defaults : [0, 0, 0, 0, 0]
String defaults : [null, null, null]
length is fixed = 5

Array[]

```
int[] a = {10, 20, 30, 40, 0, 0, 0, 0};
int size = 4;
```

~~value index~~ :	10	20	30	40	0	0	0	0
Index ~~value~~ :	0	1	2	3	4	5	6	7

a[2] = 30

ArrayStack

Initial Object

ArrayStack<Integer> s = new ArrayStack();

Capacity = 4
Initially = top = -1
Stack : [null, null, null, null]
top = -1 means empty.

push(10)

data[++top] = item;

first
top = -1 ++top: top = 0

Store 10

Index	0	1	2	3
value	10	null	null	null

Store 20
top = 1

0	1	2	3
10	20	null	null

Store 30
top = 2

0	1	2	3
10	20	30	null

PAGE No.
DATE / /

peek()

return data[top];
top = 2

data[2] = 30 , output → peek → 30

pop()
current: 10 20 30 null
↑
top.

then,
data[top--] = null;
first,
data[2] = null
then, top = 1
stack = 10 20 null null.

Push 40
top = 2
10 20 40 null

Push 50
top = 3
10 20 40 50

isFull() ← checks top == data.length - 1
3 == 4 - 1
3 == 3 → True.

Push 60
S.push(60);
If (isFull()) ← is True.

While loop
Current : 10 20 40 50

first pop : 50 , Remaining → 10 20 40
second pop : 40 , Remaining → 10 20
third pop : 20 , Remaining → 10
fourth pop : 10 , Remaining → empty.

O/P : 50 40 20 10.

BalancedBrackets

Test 1

a[i] = (b+c) * {d}

a → ignore
[→ push [
i → ignore
] → pop [→ matches
(→ push (
b → ignore
+ → ignore
c → ignore
) → pop (→ matches
{ → push {
d → ignor
} → pop {

At end:
stack = empty. → so, balanced.

Test 2

{[()()]}

{ → push {
[→ push [
(→ push (
) → pop (

(→ push (
) → pop (
] → pop [
} → pop {

Stack becomes empty → balanced.

Test 3
([)] → (→ push (
[→ push [
) → closing bracket.

Top is '[' but ')' needs '('
return false, Result = NOT balanced.

Test 4 ((a+b)
(→ push , (→ push.
then] → pop one (→ NOT balanced.

Test 4.5 → a+b)
→ only closing not opening.

NOT balanced.

Test 6
" "
↑
There are no brackets

So stack remains empty.

Balanced

PAGE No.
DATE / /

Big O Recap

- `for(int i = 0; i < n; i += 2)`

If n = 10
Values of i:

i = 0 → count 1	i = 6 → count 4
i = 2 → count 2	i = 8 → count 5
i = 4 → count 3	i = 10 → condition false

So, c = 5
For n = 1000: c = 500
For n = 2000: c = 1000
Ratio: 1000/500 = 2
So, work grows linearly.

-
```
for (int i = 0; i < n; i++)
    for (int j = 0; j < 1000; j++)
        c++;
```

If n = 10, then 10 × 1000 = 10000
For n = 1000, 1000 × 1000 = 1,000,000.
For n = 2000, 2,000,000

-
```
while (n > 0) {
    n /= 10;
    c++;
}
```

PAGE No.
DATE / /

n = 1234

Dry run:

n = 1234 → 123 → c = 1

n = 123 → 12 → c = 2

n = 12 → 1 → c = 3

n = 1 → 0 → c = 4

n = 0 → stop.

So: c = 4 → counts the number of decimal digits.

- ~~for~~ (int i = 0; i < n; i++)
  ~~for~~ (int j = 1; j < n; j++)

Take, n = 4

i = 0

j = 0, 1, 2, 3 ← 4 iterations.

i = 1

j = 1, 2, 3 ← 3 iterations.

i = 2

j = 2, 3 ← 2 iterations

i = 3

j = 3 ← 1 iterations.

Total: 4 + 3 + 2 + 1 = 10.

- for (int i = n; i > 0; i /= 2)
  for (int j = 0; j < n; j++)

Outer loops.: n
n/2
n/4
n/8
....

- for (int i = 1; i < n ; i *= 2)

Take
n = 10

Values:
i = 1
i = 2
i = 4
i = 8
i = 16 → Stop.

Inside
for (int j = 0; j < i; j++)

No. of execution : 1 + 2 + 4 + 8 = 15

- for (int i = 0; i < n; i++) → run n times
Inner: for (int j = 1; j * j <= n; j++)

- for (int i = 1; i <= n ; i++)
  for (int j = i; j <= n ; j += i)

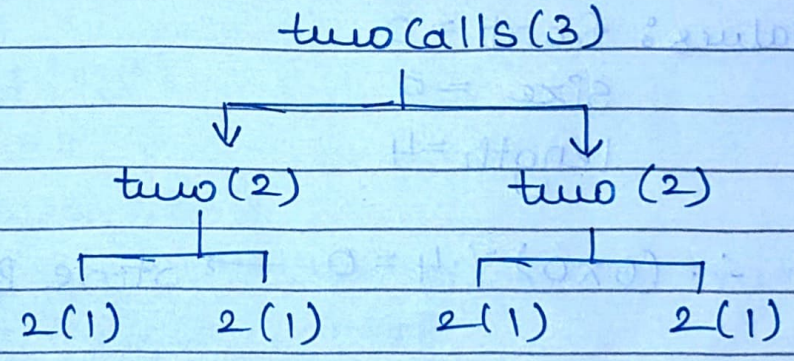
For example n = 6

i=1 → j=1, 2, 3, 4, 5, 6
i=2 → j=2, 4, 6
i=3 → j=3, 6
i=4 → j=4
i=5 → j=5
i=6 → j=6

Total : 6+3+2+1+1+1=14

- twoCalls()

For n=3



Number of calls : 7

- oneCall()

return 2* oneCall(n-1);

For, n=3

oneCall(3)
|
oneCall(2)
|
oneCall(1)

calls = 3

PAGE No.
DATE / /

Circular Deque

- CircularDeque<String> d = new CircularDeque<>(4);

So,

capacity = 4
front = 0
size = 0

Array

index:	0	1	2	3
	null	null	null	null

- addLast("B")

Values: front = 0
size = 0
length = 4

∴ (0 + 0) % 4 = 0 → Store B at index 0

[B, null, null, null]
size = 1

- addLast("C")

(0 + 1) % 4 = 1

Store C:

[B, C, null, null]
size = 2

- addFirst("A")

Current:

front = 0

PAGE No.
DATE / /

$(0-1+4) \% 4$ front = 3
$= 3-1.4$ Store A at index 3
$= 3$

[B, C, null, A]

Logical ~~order~~ is : A, B, C

addFirst("Z")
front = 3 front = 2
$(3-1+4) \% 4$ Store Z
$= 6 \% 4 = 2$ [B, C, Z, A]

addLast("D")
size = 4
data.length = 4
So, if (size == data.length) is true.
capacity becomes : 4 → 8

before : Z A B C after : [Z, A, B, C, null, null, null, null]

front = 0
addLast("D"): (front + size) % length
$(0+4) \% 8 = 4$

Deque used a Stack

Add 1 → 1
Add 2 → 2 1
Add 3 → capacity is 2.

Deque used as Queue.

Logical queue : 1, 2, 3

then, removeFirst()

returns 1

Next 2 , Next 3

as a queue : 1 2 3